Pedestrian Flow Prediction for Main Roads WA — Final Model Comparison

Yashika Yashika (24301477)

18 September 2026

Contents

1 Executive summary **2**

2 1. Setup **2**

3 2. Load the updated modelling dataset **3**

4 3. Target and feature definition **4**

5 4. Data-quality checks **6**

6 5. Response distributions **7**

7 6. Multicollinearity screening **8**

8 7. Grouped five-fold cross-validation **9**

9 8. Leakage-safe preprocessing **10**

10 9. Evaluation metrics **12**

11 10. Models compared **13**

12 11. Model helper functions **13**

13 12. Grouped five-fold model comparison **17**

14 13. Overall model ranking **20**

15 14. Improvement over the baseline **21**

16 15. Actual versus predicted **22**

17 16. Residual diagnostics **23**

18 17. Largest prediction errors **24**

19 18. Road-level bootstrap uncertainty **27**

20 19. Selected model versus runner-up **29**

21 20. Bootstrap distributions for the two leading models **30**

22 21. Fit final models to all observations **31**

23 22. Interpretable linear-model coefficients **32**

24 23. LASSO selected coefficients **33**

25 24. Random Forest feature importance **34**

26 25. Gradient Boosting feature importance **36**

27 26. Dashboard-ready out-of-fold predictions **37**

28 27. Save the final model bundle **39**

29 28. Final conclusion **40**

30 29. Key limitations **41**

31 30. Recommended next steps **41**

32 31. What to show in the next team/client meeting **41**

1 Executive summary

This notebook is the final model-development and validation workflow for the **Pedestrian Flow Prediction for Main Roads WA** capstone project.

Six approaches are compared under the same validation design:

1. Median baseline
2. Parsimonious linear regression
3. Ridge regression
4. LASSO regression
5. Random Forest
6. Gradient Boosting

Both observed pedestrian-flow targets are evaluated:

- `obs_peak_cross_ped_per_h` — peak pedestrian flow crossing the road;
- `obs_peak_along_ped_per_h` — peak pedestrian flow travelling along the road.

Because the survey dataset is small, model choice is based on **out-of-fold generalisation**, not training fit. Five-fold cross-validation is grouped by `road_no`, keeping observations from the same road in the same fold. Performance is reported using out-of-sample R-squared, RMSE and MAE. A road-level bootstrap then quantifies uncertainty in performance.

The notebook also provides data-quality checks, response distributions, multicollinearity screening, leakage-safe preprocessing, actual-versus-predicted plots, residual diagnostics, largest-error tables, feature interpretation, dashboard-ready predictions and saved final models.

2 1. Setup

```
set.seed(5553)

required_packages <- c(
  "dplyr", "tidyr", "purrr", "tibble",
  "ggplot2", "knitr", "glmnet", "ranger", "gbm", "readxl"
```

```
)

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    paste0(
      "Install the following packages before knitting: ",
      paste(missing_packages, collapse = ", ")
    )
  )
}

library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
library(knitr)
library(glmnet)
library(ranger)
library(gbm)
library(readxl)

theme_set(theme_minimal(base_size = 12))
options(dplyr.summarise.inform = FALSE)
```

3 2. Load the updated modelling dataset

The analysis uses the updated feature naming convention, including the `overture_*` variables.

```
data_path <- "../data/processed/model_dataset.xlsx"

dat <- read_excel(
  data_path,
  sheet = "Model_Data"
)

if (!"road_no" %in% names(dat)) {
  stop("The dataset must contain road_no for grouped cross-validation.")
}

required_targets <- c(
  "obs_peak_cross_ped_per_h",
  "obs_peak_along_ped_per_h"
)

missing_targets <- setdiff(required_targets, names(dat))

if (length(missing_targets) > 0) {
  stop(
```

```
    paste(
      "Missing response column(s):",
      paste(missing_targets, collapse = ", ")
    )
  )
}

if (!"site_id" %in% names(dat)) {
  dat$site_id <- seq_len(nrow(dat))
}

if (!"road_name" %in% names(dat)) {
  dat$road_name <- as.character(dat$road_no)
}

data_summary <- tibble(
  Item = c("Input file", "Observations", "Variables", "Unique roads"),
  Value = c(
    data_path,
    nrow(dat),
    ncol(dat),
    n_distinct(dat$road_no)
  )
)

kable(data_summary, caption = "Dataset summary")
```

Table 1: Dataset summary

Item	Value
Input file	../data/processed/model_dataset.xlsx
Observations	49
Variables	53
Unique roads	28

4 3. Target and feature definition

The preceding EDA indicated that commercial and retail-density variables were the strongest and most consistent predictors. Education variables are not retained because the EDA did not show a useful relationship with pedestrian counts.

Identifiers, coordinates, survey metadata and the response variables are not used as predictors.

```
if ("abs_mb_category_2021" %in% names(dat)) {
  dat <- dat %>%
    mutate(
      abs_mb_category_simple = ifelse(
        abs_mb_category_2021 %in% c("Residential", "Commercial"),
        abs_mb_category_2021,
        "Other"
      )
    )
}
```

```r
numeric_candidates <- c(
  "commercial_buildings",
  "parking_areas",
  "pedestrian_crossings",
  "overture_food_drink_count_500m",
  "overture_grocery_convenience_pharmacy_count_500m",
  "overture_other_shopping_count_500m",
  "pta_active_stops_within_500m",
  "bus_stops",
  "ms_building_count_500m",
  "abs_mb_persons_per_sqkm_2021"
)

binary_candidates <- c(
  "footpath_both_sides",
  "overture_shopping_mall_present_500m",
  "shared_pedestrian_and_cycle_path"
)

categorical_candidates <- c(
  "abs_mb_category_simple"
)

numeric_features <- intersect(numeric_candidates, names(dat))
binary_features <- intersect(binary_candidates, names(dat))
categorical_features <- intersect(categorical_candidates, names(dat))

features <- c(
  numeric_features,
  binary_features,
  categorical_features
)

if (length(features) < 2) {
  stop("Too few approved predictor columns were found in the dataset.")
}

for (v in numeric_features) {
  dat[[v]] <- suppressWarnings(
    as.numeric(gsub(",", "", dat[[v]]))
  )
}

for (v in c(binary_features, categorical_features)) {
  dat[[v]] <- as.factor(dat[[v]])
}

targets <- c(
  "Cross-road" = "obs_peak_cross_ped_per_h",
  "Along-road" = "obs_peak_along_ped_per_h"
)

feature_table <- tibble(
```

```
  Feature = features,
  Type = c(
    rep("Numeric", length(numeric_features)),
    rep("Binary", length(binary_features)),
    rep("Categorical", length(categorical_features))
  ),
  Missing = vapply(dat[features], function(x) sum(is.na(x)), numeric(1))
)

kable(feature_table, caption = "Predictors used in the final model comparison")
```

Table 2: Predictors used in the final model comparison

Feature	Type	Missing
commercial_buildings	Numeric	0
parking_areas	Numeric	0
pedestrian_crossings	Numeric	0
overture_food_drink_count_500m	Numeric	0
overture_grocery_convenience_pharmacy_count_500m	Numeric	0
overture_other_shopping_count_500m	Numeric	0
pta_active_stops_within_500m	Numeric	0
bus_stops	Numeric	49
ms_building_count_500m	Numeric	0
abs_mb_persons_per_sqkm_2021	Numeric	0
footpath_both_sides	Binary	0
overture_shopping_mall_present_500m	Binary	0
shared_pedestrian_and_cycle_path	Binary	0
abs_mb_category_simple	Categorical	0

5 4. Data-quality checks

```
target_summary <- bind_rows(
  lapply(names(targets), function(lbl) {
    y <- dat[[targets[[lbl]]]]

    tibble(
      Target = lbl,
      N = sum(!is.na(y)),
      Missing = sum(is.na(y)),
      Minimum = min(y, na.rm = TRUE),
      Median = median(y, na.rm = TRUE),
      Mean = mean(y, na.rm = TRUE),
      Maximum = max(y, na.rm = TRUE)
    )
  })
)

kable(
  target_summary,
  digits = 2,
  caption = "Response-variable summary"
)
```

Table 3: Response-variable summary

Target	N	Missing	Minimum	Median	Mean	Maximum
Cross-road	49	0	0.00	7.33	25.39	192.33
Along-road	49	0	0.67	13.00	39.87	349.67

```
missing_table <- tibble(
  Variable = c(unname(targets), features),
  Missing = vapply(
    dat[c(unname(targets), features)],
    function(x) sum(is.na(x)),
    numeric(1)
  )
) %>%
  arrange(desc(Missing))

kable(missing_table, caption = "Missing values in modelling variables")
```

Table 4: Missing values in modelling variables

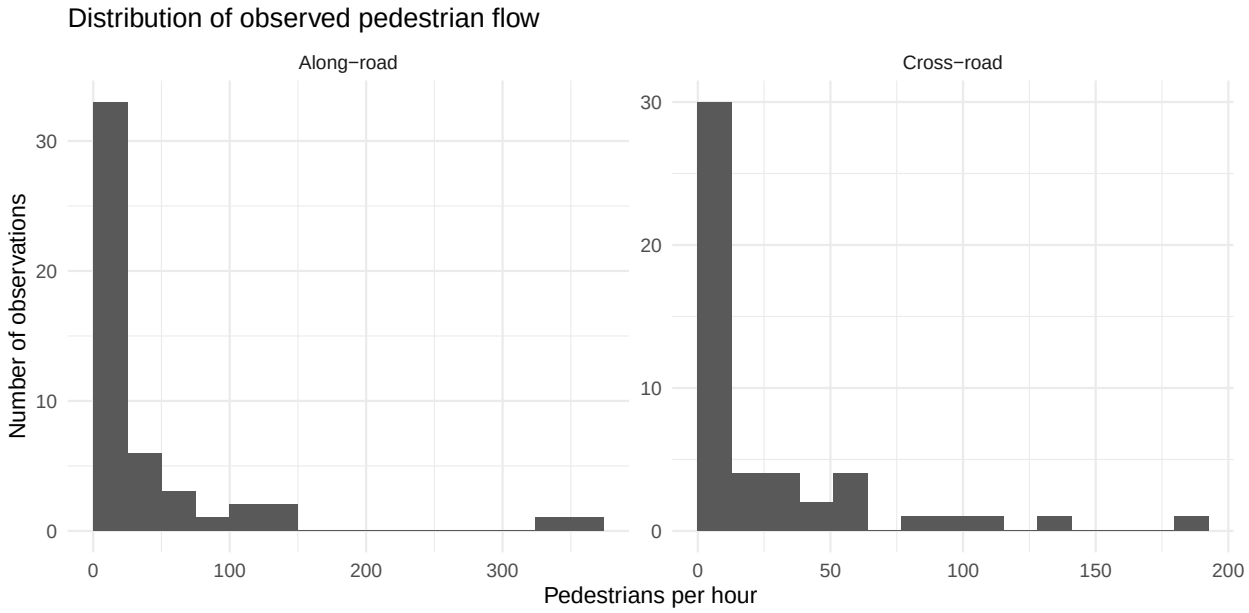
Variable	Missing
bus_stops	49
obs_peak_cross_ped_per_h	0
obs_peak_along_ped_per_h	0
commercial_buildings	0
parking_areas	0
pedestrian_crossings	0
overture_food_drink_count_500m	0
overture_grocery_convenience_pharmacy_count_500m	0
overture_other_shopping_count_500m	0
pta_active_stops_within_500m	0
ms_building_count_500m	0
abs_mb_persons_per_sqkm_2021	0
footpath_both_sides	0
overture_shopping_mall_present_500m	0
shared_pedestrian_and_cycle_path	0
abs_mb_category_simple	0

6 5. Response distributions

```
response_long <- dat %>%
  transmute(
    `Cross-road` = obs_peak_cross_ped_per_h,
    `Along-road` = obs_peak_along_ped_per_h
  ) %>%
  pivot_longer(
    everything(),
    names_to = "Target",
    values_to = "Pedestrian_flow"
```

```
  )

ggplot(response_long, aes(x = Pedestrian_flow)) +
  geom_histogram(bins = 15, boundary = 0) +
  facet_wrap(~ Target, scales = "free") +
  labs(
    title = "Distribution of observed pedestrian flow",
    x = "Pedestrians per hour",
    y = "Number of observations"
  )
```



The responses are right-skewed. Linear and regularised regression therefore use `log1p()` of the response internally, while every model is evaluated on the original pedestrians-per-hour scale.

7 6. Multicollinearity screening

Commercial and retail-density variables can be strongly correlated. The table below identifies highly correlated numeric predictor pairs. This is a diagnostic rather than an automatic deletion rule: Ridge and LASSO are included specifically because regularisation can handle correlated predictors better than unregularised regression.

```
if (length(numeric_features) >= 2) {

  cor_mat <- cor(
    dat[, numeric_features, drop = FALSE],
    use = "pairwise.complete.obs"
  )

  cor_pairs <- as.data.frame(as.table(cor_mat)) %>%
    as_tibble() %>%
    rename(
      Feature_1 = Var1,
      Feature_2 = Var2,
```

```
      Correlation = Freq
    ) %>%
    mutate(
      Feature_1 = as.character(Feature_1),
      Feature_2 = as.character(Feature_2)
    ) %>%
    filter(
      Feature_1 < Feature_2,
      abs(Correlation) >= 0.70
    ) %>%
    arrange(desc(abs(Correlation)))

  if (nrow(cor_pairs) == 0) {
    cat("No numeric predictor pairs have |correlation| >= 0.70.\n")
  } else {
    kable(
      cor_pairs,
      digits = 3,
      caption = "Highly correlated numeric predictor pairs"
    )
  }
}
```

Table 5: Highly correlated numeric predictor pairs

Feature_1	Feature_2	Correlation
overture_food_drink_count_500m	overture_grocery_convenience_pharmacy_count_500m	0.746
overture_grocery_convenience_pharmacy_count_500m	overture_other_shopping_count_500m	0.737
commercial_buildings	parking_areas	0.737
overture_food_drink_count_500m	overture_other_shopping_count_500m	0.731
commercial_buildings	overture_food_drink_count_500m	0.722

8 7. Grouped five-fold cross-validation

All observations from the same `road_no` are assigned to the same fold. This reduces leakage across segments from the same road and provides a more realistic test of prediction on unseen roads.

```
make_group_folds <- function(groups, k = 5, seed = 5553) {

  groups <- as.character(groups)
  group_sizes <- table(groups)
  unique_groups <- names(group_sizes)

  if (length(unique_groups) < 2) {
    stop("At least two unique roads are required.")
  }

  k <- min(k, length(unique_groups))

  set.seed(seed)

  group_names <- names(sort(group_sizes, decreasing = TRUE))
```

```
  tie_break <- runif(length(group_names))
  group_names <- group_names[
    order(-as.numeric(group_sizes[group_names]), tie_break)
  ]

  fold_load <- rep(0, k)
  assignment <- setNames(integer(length(group_names)), group_names)

  for (g in group_names) {
    available <- which(fold_load == min(fold_load))
    chosen <- sample(available, 1)

    assignment[g] <- chosen
    fold_load[chosen] <- fold_load[chosen] + group_sizes[g]
  }

  as.integer(assignment[groups])
}

fold_id <- make_group_folds(
  dat$road_no,
  k = 5,
  seed = 5553
)

fold_summary <- dat %>%
  mutate(Fold = fold_id) %>%
  group_by(Fold) %>%
  summarise(
    Observations = n(),
    Roads = n_distinct(road_no),
    Road_numbers = paste(sort(unique(road_no)), collapse = ", ")
  )

kable(
  fold_summary,
  caption = "Grouped five-fold cross-validation structure"
)
```

Table 6: Grouped five-fold cross-validation structure

Fold	Observations	Roads	Road_numbers
1	15	5	H009, H053, M023, M028, M074
2	10	7	H008, H042, H043, H044, H049, M002, M041
3	9	5	H001, H045, H051, M026, M031
4	7	5	H006, H007, H010, H011, M042
5	8	6	H004, H005, H046, H047, M032, M045

9 8. Leakage-safe preprocessing

Missing-value imputation is learned from the training fold only. Numeric variables use the training median. Binary and categorical variables use the training mode. New categorical levels in a validation fold are mapped

to the training-mode level.

```
mode_value <- function(x) {
  x <- as.character(x)
  x <- x[!is.na(x) & x != ""]

  if (length(x) == 0) {
    return(NA_character_)
  }

  names(sort(table(x), decreasing = TRUE))[1]
}

fit_preprocessor <- function(train) {

  medians <- if (length(numeric_features) > 0) {
    vapply(
      train[numeric_features],
      function(x) median(x, na.rm = TRUE),
      numeric(1)
    )
  } else {
    numeric(0)
  }

  factor_vars <- c(binary_features, categorical_features)

  modes <- if (length(factor_vars) > 0) {
    vapply(train[factor_vars], mode_value, character(1))
  } else {
    character(0)
  }

  levels_list <- if (length(factor_vars) > 0) {
    lapply(
      train[factor_vars],
      function(x) {
        vals <- as.character(x)
        vals <- vals[!is.na(vals) & vals != ""]
        unique(vals)
      }
    )
  } else {
    list()
  }

  list(
    medians = medians,
    modes = modes,
    levels = levels_list
  )
}

apply_preprocessor <- function(df, prep) {
```

```
  out <- df

  for (v in numeric_features) {
    replacement <- prep$medians[[v]]

    if (is.na(replacement)) {
      replacement <- 0
    }

    out[[v]][is.na(out[[v]])] <- replacement
  }

  for (v in c(binary_features, categorical_features)) {

    x <- as.character(out[[v]])
    replacement <- prep$modes[[v]]
    known <- prep$levels[[v]]

    if (length(known) == 0 || is.na(replacement)) {
      known <- "Missing"
      replacement <- "Missing"
    }

    x[is.na(x) | x == ""] <- replacement
    x[!x %in% known] <- replacement

    out[[v]] <- factor(x, levels = known)
  }

  out
}
```

10 9. Evaluation metrics

R-squared is an out-of-sample goodness-of-fit measure, not percentage accuracy. RMSE penalises large errors more strongly, while MAE gives an interpretable average absolute error in pedestrians per hour.

```
metrics <- function(actual, predicted) {

  keep <- is.finite(actual) & is.finite(predicted)
  actual <- actual[keep]
  predicted <- pmax(predicted[keep], 0)

  sse <- sum((actual - predicted)^2)
  sst <- sum((actual - mean(actual))^2)

  tibble(
    R2 = ifelse(sst > 0, 1 - sse / sst, NA_real_),
    RMSE = sqrt(mean((actual - predicted)^2)),
    MAE = mean(abs(actual - predicted)),
    Median_AE = median(abs(actual - predicted))
  )
}
```

11 10. Models compared

The same grouped outer folds are used for all six approaches.

- **Median baseline:** minimum practical benchmark.
- **Linear regression:** parsimonious, interpretable EDA-informed model.
- **Ridge:** regularised regression that is useful when predictors are correlated.
- **LASSO:** regularised regression that can shrink weak predictors to zero.
- **Random Forest:** non-linear model that captures interactions and threshold effects.
- **Gradient Boosting:** second non-linear tree-based benchmark.

```
linear_features <- intersect(
  c(
    "commercial_buildings",
    "footpath_both_sides",
    "abs_mb_category_simple"
  ),
  features
)

if (length(linear_features) == 0) {
  linear_features <- features[1]
}

linear_features
```

```
## [1] "commercial_buildings"   "footpath_both_sides"    "abs_mb_category_simple"
```

12 11. Model helper functions

```
make_design_matrix <- function(df) {
  model.matrix(
    ~ . - 1,
    data = df[, features, drop = FALSE]
  )
}
```

```
fit_linear <- function(train, target) {

  prep <- fit_preprocessor(train)
  train_p <- apply_preprocessor(train, prep)

  usable_features <- linear_features[
    vapply(
      train_p[linear_features],
      function(x) length(unique(x[!is.na(x)])) > 1,
      logical(1)
    )
  ]

  if (length(usable_features) == 0) {
    stop("No varying predictors are available for the linear model in this fold.")
  }

  formula <- as.formula(
```

```
    paste0(
      "log1p(", target, ") ~ ",
      paste(usable_features, collapse = " + ")
    )
  )

  list(
    model = lm(formula, data = train_p),
    prep = prep,
    features = usable_features
  )
}

predict_linear <- function(fit, newdata) {

  x <- apply_preprocessor(newdata, fit$prep)

  pmax(
    expm1(as.numeric(predict(fit$model, newdata = x))),
    0
  )
}
```

```
fit_regularised <- function(train, target, alpha_value, seed = 5553) {

  prep <- fit_preprocessor(train)
  train_p <- apply_preprocessor(train, prep)

  x <- make_design_matrix(train_p)
  y <- log1p(train_p[[target]])

  inner_fold_id <- make_group_folds(
    train_p$road_no,
    k = min(5, n_distinct(train_p$road_no)),
    seed = seed
  )

  cv_fit <- cv.glmnet(
    x = x,
    y = y,
    alpha = alpha_value,
    foldid = inner_fold_id,
    standardize = TRUE,
    family = "gaussian"
  )

  list(
    model = cv_fit,
    prep = prep,
    matrix_columns = colnames(x)
  )
}

predict_regularised <- function(fit, newdata) {
```

```
  xdata <- apply_preprocessor(newdata, fit$prep)
  x <- make_design_matrix(xdata)

  missing_cols <- setdiff(fit$matrix_columns, colnames(x))

  if (length(missing_cols) > 0) {
    zero_mat <- matrix(
      0,
      nrow = nrow(x),
      ncol = length(missing_cols),
      dimnames = list(NULL, missing_cols)
    )
    x <- cbind(x, zero_mat)
  }

  extra_cols <- setdiff(colnames(x), fit$matrix_columns)

  if (length(extra_cols) > 0) {
    x <- x[, setdiff(colnames(x), extra_cols), drop = FALSE]
  }

  x <- x[, fit$matrix_columns, drop = FALSE]

  pmax(
    expm1(
      as.numeric(
        predict(
          fit$model,
          newx = x,
          s = "lambda.1se"
        )
      )
    ),
    0
  )
}
```

```
fit_rf <- function(train, target, seed = 5553) {

  prep <- fit_preprocessor(train)
  train_p <- apply_preprocessor(train, prep)

  formula <- as.formula(
    paste(target, "~", paste(features, collapse = " + "))
  )

  model <- ranger(
    formula,
    data = train_p[, c(target, features), drop = FALSE],
    num.trees = 1000,
    mtry = max(1, floor(sqrt(length(features)))),
    min.node.size = 4,
    importance = "permutation",
```

```
    seed = seed
  )

  list(model = model, prep = prep)
}

predict_rf <- function(fit, newdata) {

  x <- apply_preprocessor(newdata, fit$prep)

  pmax(
    predict(
      fit$model,
      data = x[, features, drop = FALSE]
    )$predictions,
    0
  )
}
```

```
fit_gb <- function(train, target, seed = 5553) {

  set.seed(seed)

  prep <- fit_preprocessor(train)
  train_p <- apply_preprocessor(train, prep)

  formula <- as.formula(
    paste(target, "~", paste(features, collapse = " + "))
  )

  model <- gbm(
    formula = formula,
    data = train_p[, c(target, features), drop = FALSE],
    distribution = "gaussian",
    n.trees = 1000,
    interaction.depth = 2,
    shrinkage = 0.02,
    n.minobsinnode = 5,
    bag.fraction = 0.8,
    train.fraction = 1.0,
    verbose = FALSE
  )

  list(model = model, prep = prep)
}

predict_gb <- function(fit, newdata) {

  x <- apply_preprocessor(newdata, fit$prep)

  pmax(
    as.numeric(
      predict(
        fit$model,
```

```
        newdata = x[, features, drop = FALSE],
        n.trees = 1000
      )
    ),
    0
  )
}
```

13 12. Grouped five-fold model comparison

Every prediction below is out of fold: the model producing it was not trained on that observation's road.

```
compare_models_cv <- function(target, target_label) {

  n <- nrow(dat)

  prediction_store <- list(
    "Median baseline" = rep(NA_real_, n),
    "Linear regression" = rep(NA_real_, n),
    "Ridge" = rep(NA_real_, n),
    "LASSO" = rep(NA_real_, n),
    "Random Forest" = rep(NA_real_, n),
    "Gradient Boosting" = rep(NA_real_, n)
  )

  for (fold in sort(unique(fold_id))) {

    train_idx <- fold_id != fold
    test_idx <- fold_id == fold

    train <- dat[train_idx, , drop = FALSE]
    test <- dat[test_idx, , drop = FALSE]

    train <- train[!is.na(train[[target]]), , drop = FALSE]
    valid_test <- !is.na(test[[target]])

    if (nrow(train) < 5 || sum(valid_test) == 0) {
      next
    }

    test_valid <- test[valid_test, , drop = FALSE]
    original_indices <- which(test_idx)[valid_test]

    prediction_store[["Median baseline"]][original_indices] <-
      median(train[[target]], na.rm = TRUE)

    lm_fit <- fit_linear(train, target)
    prediction_store[["Linear regression"]][original_indices] <-
      predict_linear(lm_fit, test_valid)

    ridge_fit <- fit_regularised(
      train,
      target,
```

```
    alpha_value = 0,
    seed = 5553 + fold
  )
  prediction_store[["Ridge"]][original_indices] <-
    predict_regularised(ridge_fit, test_valid)

  lasso_fit <- fit_regularised(
    train,
    target,
    alpha_value = 1,
    seed = 6553 + fold
  )
  prediction_store[["LASSO"]][original_indices] <-
    predict_regularised(lasso_fit, test_valid)

  rf_fit <- fit_rf(
    train,
    target,
    seed = 7553 + fold
  )
  prediction_store[["Random Forest"]][original_indices] <-
    predict_rf(rf_fit, test_valid)

  gb_fit <- fit_gb(
    train,
    target,
    seed = 8553 + fold
  )
  prediction_store[["Gradient Boosting"]][original_indices] <-
    predict_gb(gb_fit, test_valid)
}

prediction_table <- map_dfr(
  names(prediction_store),
  function(model_name) {
    tibble(
      site_id = dat$site_id,
      road_no = dat$road_no,
      road_name = dat$road_name,
      actual = dat[[target]],
      predicted = prediction_store[[model_name]],
      Model = model_name,
      Target = target_label
    )
  }
) %>%
  filter(is.finite(actual), is.finite(predicted)) %>%
  mutate(
    residual = actual - predicted,
    absolute_error = abs(residual)
  )

performance <- prediction_table %>%
```

```
    group_by(Target, Model) %>%
    group_modify(~ metrics(.x$actual, .x$predicted)) %>%
    ungroup()

  list(
    predictions = prediction_table,
    performance = performance
  )
}

cross_results <- compare_models_cv(
  targets[["Cross-road"]],
  "Cross-road"
)
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
```

```
along_results <- compare_models_cv(
  targets[["Along-road"]],
  "Along-road"
)
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
```

```
all_predictions <- bind_rows(
  cross_results$predictions,
  along_results$predictions
)

cv_performance <- bind_rows(
  cross_results$performance,
  along_results$performance
)

kable(
  cv_performance %>% arrange(Target, desc(R2)),
  digits = 3,
```

```
  caption = "Grouped five-fold cross-validated model performance"
)
```

Table 7: Grouped five-fold cross-validated model performance

Target	Model	R2	RMSE	MAE	Median_AE
Along-road	Linear regression	0.719	37.017	20.261	8.836
Along-road	Random Forest	0.327	57.252	30.841	14.996
Along-road	Gradient Boosting	0.237	60.965	34.281	14.789
Along-road	Ridge	-0.074	72.336	31.858	9.360
Along-road	Median baseline	-0.156	75.044	32.935	9.333
Along-road	LASSO	-2.465	129.936	43.760	9.009
Cross-road	Linear regression	0.425	29.138	15.181	4.740
Cross-road	Random Forest	0.380	30.235	19.055	10.265
Cross-road	Gradient Boosting	0.045	37.532	23.074	10.189
Cross-road	LASSO	-0.077	39.873	21.582	7.847
Cross-road	Ridge	-0.129	40.814	21.796	6.888
Cross-road	Median baseline	-0.227	42.541	22.874	7.167

14 13. Overall model ranking

The final model is not selected from one metric alone. Each model is ranked within each target on R-squared, RMSE and MAE, then averaged across both targets.

```
ranking_table <- cv_performance %>%
  group_by(Target) %>%
  mutate(
    Rank_R2 = rank(-R2, ties.method = "average"),
    Rank_RMSE = rank(RMSE, ties.method = "average"),
    Rank_MAE = rank(MAE, ties.method = "average"),
    Target_average_rank = rowMeans(
      cbind(Rank_R2, Rank_RMSE, Rank_MAE)
    )
  ) %>%
  ungroup()

overall_ranking <- ranking_table %>%
  group_by(Model) %>%
  summarise(
    Average_rank = mean(Target_average_rank, na.rm = TRUE),
    Mean_R2 = mean(R2, na.rm = TRUE),
    Mean_RMSE = mean(RMSE, na.rm = TRUE),
    Mean_MAE = mean(MAE, na.rm = TRUE)
  ) %>%
  arrange(Average_rank, desc(Mean_R2), Mean_RMSE)

kable(
  overall_ranking,
  digits = 3,
  caption = "Overall out-of-sample model ranking"
)
```

Table 8: Overall out-of-sample model ranking

Model	Average_rank	Mean_R2	Mean_RMSE	Mean_MAE
Linear regression	1.000	0.572	33.077	17.721
Random Forest	2.000	0.354	43.743	24.948
Gradient Boosting	3.833	0.141	49.248	28.678
Ridge	4.167	-0.101	56.575	26.827
LASSO	4.833	-1.271	84.904	32.671
Median baseline	5.167	-0.191	58.793	27.905

```
selected_model <- overall_ranking$Model[1]
runner_up_model <- overall_ranking$Model[2]

cat("Selected model from grouped-CV ranking:", selected_model, "\n")
```

```
## Selected model from grouped-CV ranking: Linear regression
```

```
cat("Runner-up:", runner_up_model, "\n")
```

```
## Runner-up: Random Forest
```

15 14. Improvement over the baseline

```
baseline_metrics <- cv_performance %>%
  filter(Model == "Median baseline") %>%
  select(
    Target,
    Baseline_RMSE = RMSE,
    Baseline_MAE = MAE
  )

improvement_table <- cv_performance %>%
  filter(Model != "Median baseline") %>%
  left_join(baseline_metrics, by = "Target") %>%
  mutate(
    RMSE_improvement_pct =
      100 * (Baseline_RMSE - RMSE) / Baseline_RMSE,
    MAE_improvement_pct =
      100 * (Baseline_MAE - MAE) / Baseline_MAE
  ) %>%
  select(
    Target, Model, R2, RMSE, MAE,
    RMSE_improvement_pct,
    MAE_improvement_pct
  )

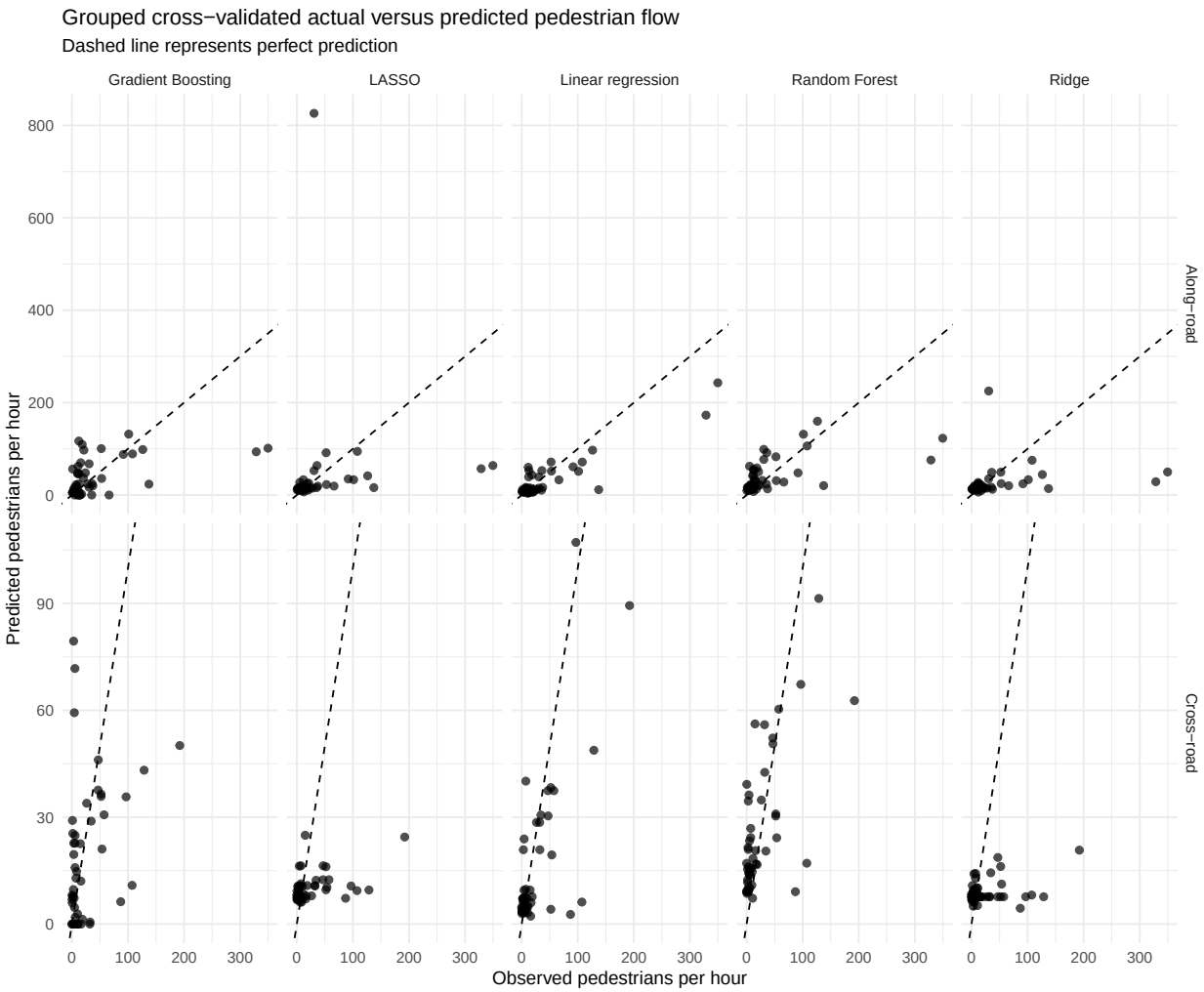
kable(
  improvement_table,
  digits = 2,
  caption = "Improvement relative to the median baseline"
)
```

Table 9: Improvement relative to the median baseline

Target	Model	R2	RMSE	MAE	RMSE_improvement_pct	MAE_improvement_pct
Cross-road	Gradient Boosting	0.05	37.53	23.07	11.78	-0.88
Cross-road	LASSO	-0.08	39.87	21.58	6.27	5.65
Cross-road	Linear regression	0.42	29.14	15.18	31.51	33.63
Cross-road	Random Forest	0.38	30.23	19.05	28.93	16.70
Cross-road	Ridge	-0.13	40.81	21.80	4.06	4.71
Along-road	Gradient Boosting	0.24	60.96	34.28	18.76	-4.08
Along-road	LASSO	-2.47	129.94	43.76	-73.15	-32.87
Along-road	Linear regression	0.72	37.02	20.26	50.67	38.48
Along-road	Random Forest	0.33	57.25	30.84	23.71	6.36
Along-road	Ridge	-0.07	72.34	31.86	3.61	3.27

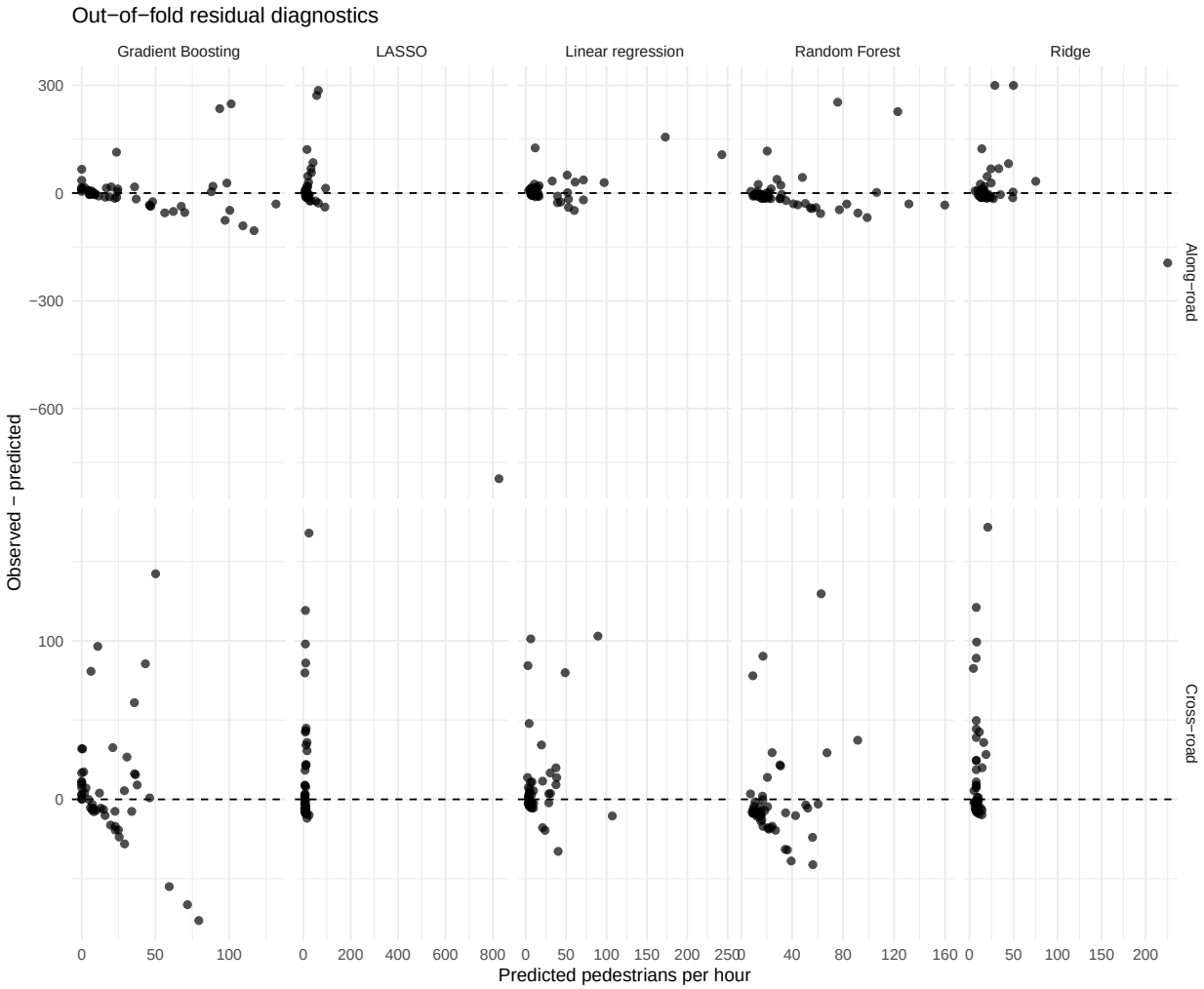
16 15. Actual versus predicted

```
ggplot(
  all_predictions %>% filter(Model != "Median baseline"),
  aes(x = actual, y = predicted)
) +
  geom_point(alpha = 0.70, size = 2) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed") +
  facet_grid(Target ~ Model, scales = "free") +
  labs(
    title = "Grouped cross-validated actual versus predicted pedestrian flow",
    subtitle = "Dashed line represents perfect prediction",
    x = "Observed pedestrians per hour",
    y = "Predicted pedestrians per hour"
  )
```



17 16. Residual diagnostics

```
ggplot(
  all_predictions %>% filter(Model != "Median baseline"),
  aes(x = predicted, y = residual)
) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point(alpha = 0.70, size = 2) +
  facet_grid(Target ~ Model, scales = "free") +
  labs(
    title = "Out-of-fold residual diagnostics",
    x = "Predicted pedestrians per hour",
    y = "Observed - predicted"
  )
```



18 17. Largest prediction errors

Large errors matter because a small number of high-flow sites can strongly affect RMSE.

```
largest_errors <- all_predictions %>%
  filter(Model != "Median baseline") %>%
  group_by(Target, Model) %>%
  arrange(desc(absolute_error), .by_group = TRUE) %>%
  slice_head(n = 5) %>%
  ungroup() %>%
  select(
    Target, Model,
    site_id, road_no, road_name,
    actual, predicted,
    residual, absolute_error
  )

kable(
  largest_errors,
  digits = 2,
  caption = "Five largest out-of-fold errors for each model and target"
```

)

Table 10: Five largest out-of-fold errors for each model and target

Target	Model	site_id	road_no	road_name	actual	predicted	residual	absolute_error
Along-road	Gradient Boosting	250303-10	H009	South Western Hwy	349.67	101.36	248.31	248.31
Along-road	Gradient Boosting	250303-11	H043	Bussell Hwy	328.67	93.60	235.06	235.06
Along-road	Gradient Boosting	260335-33	H001	Albany Hwy	137.33	23.57	113.77	113.77
Along-road	Gradient Boosting	250303-13	H009	South Western Hwy	12.33	116.88	-104.54	104.54
Along-road	Gradient Boosting	260335-38	H044	Carnarvon Rd	18.33	109.34	-91.00	91.00
Along-road	LASSO	250303-16	M023	Pinjarra Rd	30.67	826.00	-795.33	795.33
Along-road	LASSO	250303-10	H009	South Western Hwy	349.67	63.86	285.81	285.81
Along-road	LASSO	250303-11	H043	Bussell Hwy	328.67	56.97	271.69	271.69
Along-road	LASSO	260335-33	H001	Albany Hwy	137.33	16.05	121.29	121.29
Along-road	LASSO	260335-35	H009	South Western Hwy	126.33	41.54	84.80	84.80
Along-road	Linear regression	250303-11	H043	Bussell Hwy	328.67	172.84	155.83	155.83
Along-road	Linear regression	260335-33	H001	Albany Hwy	137.33	11.59	125.74	125.74
Along-road	Linear regression	250303-10	H009	South Western Hwy	349.67	242.89	106.78	106.78
Along-road	Linear regression	250303-03	H045	Coalfields Hwy	101.33	51.29	50.04	50.04
Along-road	Linear regression	260335-47	M031	Great Southern Hwy	11.67	60.11	-48.44	48.44
Along-road	Random Forest	250303-11	H043	Bussell Hwy	328.67	75.70	252.97	252.97
Along-road	Random Forest	250303-10	H009	South Western Hwy	349.67	122.88	226.79	226.79
Along-road	Random Forest	260335-33	H001	Albany Hwy	137.33	20.37	116.96	116.96
Along-road	Random Forest	250303-16	M023	Pinjarra Rd	30.67	98.81	-68.15	68.15
Along-road	Random Forest	250303-15	M074	Lakelands Lake Clifton	5.33	62.28	-56.95	56.95
Along-road	Ridge	250303-11	H043	Bussell Hwy	328.67	28.79	299.88	299.88
Along-road	Ridge	250303-10	H009	South Western Hwy	349.67	49.90	299.76	299.76
Along-road	Ridge	250303-16	M023	Pinjarra Rd	30.67	225.12	-194.45	194.45
Along-road	Ridge	260335-33	H001	Albany Hwy	137.33	13.96	123.37	123.37

Target	Model	site_id	road_no	road_name	actual	predicted	residual	absolute_error
Along-road	Ridge	260335-35	H009	South Western Hwy	126.33	44.32	82.01	82.01
Cross-road	Gradient Boosting	250303-11	H043	Bussell Hwy	192.33	50.12	142.21	142.21
Cross-road	Gradient Boosting	260335-33	H001	Albany Hwy	107.33	10.88	96.45	96.45
Cross-road	Gradient Boosting	260335-35	H009	South Western Hwy	128.67	43.20	85.46	85.46
Cross-road	Gradient Boosting	250341-04	H006	GNH	87.00	6.28	80.72	80.72
Cross-road	Gradient Boosting	250303-13	H009	South Western Hwy	3.00	79.42	-76.42	76.42
Cross-road	LASSO	250303-11	H043	Bussell Hwy	192.33	24.44	167.89	167.89
Cross-road	LASSO	260335-35	H009	South Western Hwy	128.67	9.57	119.10	119.10
Cross-road	LASSO	260335-33	H001	Albany Hwy	107.33	9.40	97.94	97.94
Cross-road	LASSO	250303-10	H009	South Western Hwy	96.67	10.69	85.97	85.97
Cross-road	LASSO	250341-04	H006	GNH	87.00	7.25	79.75	79.75
Cross-road	Linear regression	250303-11	H043	Bussell Hwy	192.33	89.41	102.92	102.92
Cross-road	Linear regression	260335-33	H001	Albany Hwy	107.33	6.18	101.15	101.15
Cross-road	Linear regression	250341-04	H006	GNH	87.00	2.71	84.29	84.29
Cross-road	Linear regression	260335-35	H009	South Western Hwy	128.67	48.79	79.88	79.88
Cross-road	Linear regression	250303-06	H009	South Western Hwy	52.00	4.16	47.84	47.84
Cross-road	Random Forest	250303-11	H043	Bussell Hwy	192.33	62.72	129.61	129.61
Cross-road	Random Forest	260335-33	H001	Albany Hwy	107.33	17.08	90.25	90.25
Cross-road	Random Forest	250341-04	H006	GNH	87.00	9.09	77.91	77.91
Cross-road	Random Forest	250303-16	M023	Pinjarra Rd	15.00	56.19	-41.19	41.19
Cross-road	Random Forest	250303-15	M074	Lakelands Lake Clifton	0.33	39.22	-38.89	38.89
Cross-road	Ridge	250303-11	H043	Bussell Hwy	192.33	20.76	171.57	171.57
Cross-road	Ridge	260335-35	H009	South Western Hwy	128.67	7.67	121.00	121.00
Cross-road	Ridge	260335-33	H001	Albany Hwy	107.33	8.15	99.18	99.18
Cross-road	Ridge	250303-10	H009	South Western Hwy	96.67	7.67	89.00	89.00
Cross-road	Ridge	250341-04	H006	GNH	87.00	4.44	82.56	82.56

19 18. Road-level bootstrap uncertainty

Cross-validation provides the primary performance estimates. The bootstrap below resamples complete roads from the already out-of-fold predictions to quantify uncertainty; it does not replace cross-validation.

```
bootstrap_models <- function(prediction_data, B = 1000, seed = 5553) {

  set.seed(seed)

  roads <- unique(prediction_data$road_no)
  models <- unique(prediction_data$Model)

  map_dfr(seq_len(B), function(b) {

    sampled_roads <- sample(
      roads,
      size = length(roads),
      replace = TRUE
    )

    sampled <- map_dfr(
      seq_along(sampled_roads),
      function(i) {
        prediction_data %>%
          filter(road_no == sampled_roads[i]) %>%
          mutate(Bootstrap_cluster = i)
      }
    )

    map_dfr(models, function(m) {
      x <- sampled %>% filter(Model == m)

      metrics(x$actual, x$predicted) %>%
        mutate(
          Bootstrap = b,
          Model = m,
          .before = 1
        )
    })
  })
}

cross_boot <- bootstrap_models(
  all_predictions %>% filter(Target == "Cross-road"),
  B = 1000,
  seed = 5553
) %>%
  mutate(Target = "Cross-road")

along_boot <- bootstrap_models(
  all_predictions %>% filter(Target == "Along-road"),
  B = 1000,
  seed = 5554
) %>%
```

```r
  mutate(Target = "Along-road")

bootstrap_results <- bind_rows(cross_boot, along_boot)

bootstrap_ci <- bootstrap_results %>%
  group_by(Target, Model) %>%
  summarise(
    R2_median = median(R2, na.rm = TRUE),
    R2_low = quantile(R2, 0.025, na.rm = TRUE),
    R2_high = quantile(R2, 0.975, na.rm = TRUE),
    RMSE_median = median(RMSE, na.rm = TRUE),
    RMSE_low = quantile(RMSE, 0.025, na.rm = TRUE),
    RMSE_high = quantile(RMSE, 0.975, na.rm = TRUE),
    MAE_median = median(MAE, na.rm = TRUE),
    MAE_low = quantile(MAE, 0.025, na.rm = TRUE),
    MAE_high = quantile(MAE, 0.975, na.rm = TRUE)
  )

kable(
  bootstrap_ci,
  digits = 3,
  caption = "Road-level bootstrap 95% intervals"
)
```

Table 11: Road-level bootstrap 95% intervals

Target	Model	R2_median	R2_low	R2_high	RMSE_median	RMSE_low	RMSE_high	MAE_median	MAE_low	MAE_high
Along-road	Gradient Boosting	0.220	-1.364	0.342	59.580	27.428	79.652	33.344	18.105	47.434
Along-road	LASSO	-2.274	-59.340	0.248	128.167	30.040	222.952	41.993	16.157	85.108
Along-road	Linear regression	0.711	-0.083	0.840	36.103	15.616	51.267	19.583	10.499	28.624
Along-road	Median baseline	-0.157	-0.290	-0.057	73.328	20.556	102.273	31.611	12.577	50.602
Along-road	Random Forest	0.302	-0.721	0.454	55.659	23.468	78.292	29.963	16.282	42.922
Along-road	Ridge	-0.071	-3.411	0.211	70.510	29.450	96.265	31.189	14.847	48.780
Cross-road	Gradient Boosting	0.012	-0.979	0.245	36.788	23.474	48.398	22.798	14.364	29.527
Cross-road	LASSO	-0.078	-0.321	0.103	39.014	17.615	53.828	21.026	9.495	30.024
Cross-road	Linear regression	0.410	-0.135	0.635	28.657	16.745	38.740	14.859	7.992	21.001
Cross-road	Median baseline	-0.220	-0.478	-0.040	41.454	18.093	57.920	22.325	9.686	32.306
Cross-road	Random Forest	0.353	-0.429	0.619	29.514	18.502	43.111	18.811	13.740	26.195
Cross-road	Ridge	-0.126	-0.398	0.085	39.867	17.648	54.943	21.191	9.207	30.526

20 19. Selected model versus runner-up

For RMSE and MAE, a negative `Selected - Runner-up` difference favours the selected model. For R-squared, a positive difference favours the selected model.

```
top_two_boot <- bootstrap_results %>%
  filter(Model %in% c(selected_model, runner_up_model)) %>%
  mutate(Model_key = make.names(Model)) %>%
  select(
    Target, Bootstrap, Model_key,
    R2, RMSE, MAE
  ) %>%
  pivot_wider(
    names_from = Model_key,
    values_from = c(R2, RMSE, MAE)
  )

selected_key <- make.names(selected_model)
runner_key <- make.names(runner_up_model)

selected_runner_diff <- top_two_boot %>%
  mutate(
    R2_difference =
      .data[[paste0("R2_", selected_key)]] -
      .data[[paste0("R2_", runner_key)]],
    RMSE_difference =
      .data[[paste0("RMSE_", selected_key)]] -
      .data[[paste0("RMSE_", runner_key)]],
    MAE_difference =
      .data[[paste0("MAE_", selected_key)]] -
      .data[[paste0("MAE_", runner_key)]]
  ) %>%
  group_by(Target) %>%
  summarise(
    R2_diff_median = median(R2_difference, na.rm = TRUE),
    R2_diff_low = quantile(R2_difference, 0.025, na.rm = TRUE),
    R2_diff_high = quantile(R2_difference, 0.975, na.rm = TRUE),
    RMSE_diff_median = median(RMSE_difference, na.rm = TRUE),
    RMSE_diff_low = quantile(RMSE_difference, 0.025, na.rm = TRUE),
    RMSE_diff_high = quantile(RMSE_difference, 0.975, na.rm = TRUE),
    MAE_diff_median = median(MAE_difference, na.rm = TRUE),
    MAE_diff_low = quantile(MAE_difference, 0.025, na.rm = TRUE),
    MAE_diff_high = quantile(MAE_difference, 0.975, na.rm = TRUE)
  )

kable(
  selected_runner_diff,
  digits = 3,
  caption = paste(
    "Bootstrap differences:",
    selected_model,
    "minus",
    runner_up_model
  )
```

```
)
```

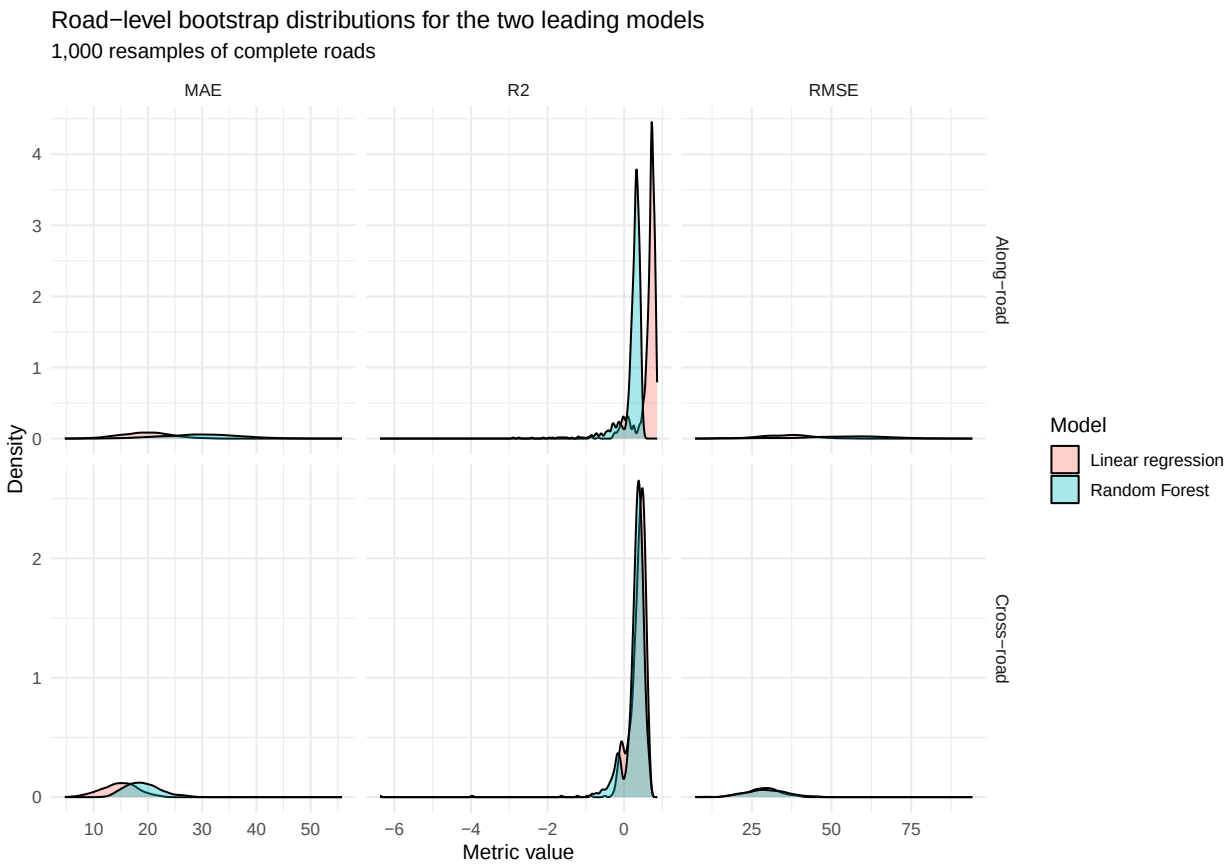
Table 12: Bootstrap differences: Linear regression minus Random Forest

Target	R2_diff_median	R2_diff_low	R2_diff_high	RMSE_diff_median	RMSE_diff_low	RMSE_diff_high	MAE_diff_median	MAE_diff_low	MAE_diff_high
Along-road	0.402	0.061	0.793	-20.198	-29.842	-1.061	-10.574	-16.228	-3.235
Cross-road	0.059	-0.201	0.465	-1.382	-8.213	4.450	-4.246	-9.257	-0.055

21 20. Bootstrap distributions for the two leading models

```
bootstrap_plot_data <- bootstrap_results %>%
  filter(Model %in% c(selected_model, runner_up_model)) %>%
  select(Target, Model, Bootstrap, R2, RMSE, MAE) %>%
  pivot_longer(
    c(R2, RMSE, MAE),
    names_to = "Metric",
    values_to = "Value"
  )

ggplot(
  bootstrap_plot_data,
  aes(x = Value, fill = Model)
) +
  geom_density(alpha = 0.35) +
  facet_grid(Target ~ Metric, scales = "free") +
  labs(
    title = "Road-level bootstrap distributions for the two leading models",
    subtitle = "1,000 resamples of complete roads",
    x = "Metric value",
    y = "Density",
    fill = "Model"
  )
```



22 21. Fit final models to all observations

Cross-validation is used for evaluation. After evaluation, each model is refitted to all approved observations for interpretation and later network-wide prediction.

```
fit_final_model_set <- function(target, seed_base = 5553) {

  train <- dat[!is.na(dat[[target]]), , drop = FALSE]

  list(
    linear = fit_linear(train, target),
    ridge = fit_regularised(
      train, target,
      alpha_value = 0,
      seed = seed_base
    ),
    lasso = fit_regularised(
      train, target,
      alpha_value = 1,
      seed = seed_base + 100
    ),
    random_forest = fit_rf(
      train, target,
      seed = seed_base + 200
    ),
```

```
    gradient_boosting = fit_gb(
      train, target,
      seed = seed_base + 300
    )
  )
}

final_cross <- fit_final_model_set(
  targets[["Cross-road"]],
  seed_base = 5553
)
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
```

```
final_along <- fit_final_model_set(
  targets[["Along-road"]],
  seed_base = 6553
)
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 8: bus_stops has no variation.
```

23 22. Interpretable linear-model coefficients

```
linear_coefficient_table <- function(fit, target_label) {

  coef_df <- as.data.frame(summary(fit$model)$coefficients)
  coef_df$Term <- rownames(coef_df)
  rownames(coef_df) <- NULL

  as_tibble(coef_df) %>%
    transmute(
      Target = target_label,
      Term = Term,
      Estimate = Estimate,
      Multiplicative_effect = exp(Estimate),
      Std_Error = `Std. Error`,
      P_value = `Pr(>|t|)`
    )
}

linear_coefficients <- bind_rows(
  linear_coefficient_table(final_cross$linear, "Cross-road"),
  linear_coefficient_table(final_along$linear, "Along-road")
)

kable(
  linear_coefficients,
  digits = 4,
  caption = "Parsimonious linear-model coefficients"
)
```

Table 13: Parsimonious linear-model coefficients

Target	Term	Estimate	Multiplicative_effect	Std_Error	P_value
Cross-road	(Intercept)	1.4403	4.2218	0.2329	0.0000
Cross-road	commercial_buildings	0.2616	1.2990	0.0728	0.0008
Cross-road	footpath_both_sidesTRUE	0.4261	1.5313	0.3854	0.2749
Cross-road	abs_mb_category_simpleOther	0.1676	1.1825	0.3532	0.6374
Cross-road	abs_mb_category_simpleCommercial	0.0724	1.0751	0.4651	0.8769
Along-road	(Intercept)	1.9802	7.2445	0.1706	0.0000
Along-road	commercial_buildings	0.2962	1.3447	0.0533	0.0000
Along-road	footpath_both_sidesTRUE	0.2727	1.3135	0.2822	0.3393
Along-road	abs_mb_category_simpleOther	0.1193	1.1267	0.2587	0.6469
Along-road	abs_mb_category_simpleCommercial	-0.1073	0.8982	0.3406	0.7542

The coefficient table is included for interpretation only. Predictive model selection is based on grouped cross-validation rather than p-values or in-sample fit.

24 23. LASSO selected coefficients

```
extract_lasso <- function(fit, target_label) {

  b <- as.matrix(
    coef(fit$model, s = "lambda.1se")
  )

  tibble(
    Target = target_label,
    Feature = rownames(b),
    Coefficient = as.numeric(b[, 1])
  ) %>%
    filter(
      Feature != "(Intercept)",
      Coefficient != 0
    ) %>%
    arrange(desc(abs(Coefficient)))
}

lasso_coefficients <- bind_rows(
  extract_lasso(final_cross$lasso, "Cross-road"),
  extract_lasso(final_along$lasso, "Along-road")
)

kable(
  lasso_coefficients,
  digits = 4,
  caption = "Non-zero LASSO coefficients at lambda.1se"
)
```

Table 14: Non-zero LASSO coefficients at lambda.1se

Target	Feature	Coefficient
Cross-road	commercial_buildings	0.1470
Along-road	commercial_buildings	0.1776

25 24. Random Forest feature importance

Permutation importance measures predictive contribution, not causality.

```
rf_importance <- function(fit, target_label) {

  x <- fit$model$variable.importance

  tibble(
    Target = target_label,
    Feature = names(x),
    Importance = as.numeric(x)
  ) %>%
    arrange(desc(Importance))
}

rf_imp <- bind_rows(
  rf_importance(final_cross$random_forest, "Cross-road"),
  rf_importance(final_along$random_forest, "Along-road")
)

rf_top <- rf_imp %>%
  group_by(Target) %>%
  slice_max(Importance, n = 10, with_ties = FALSE) %>%
  ungroup()

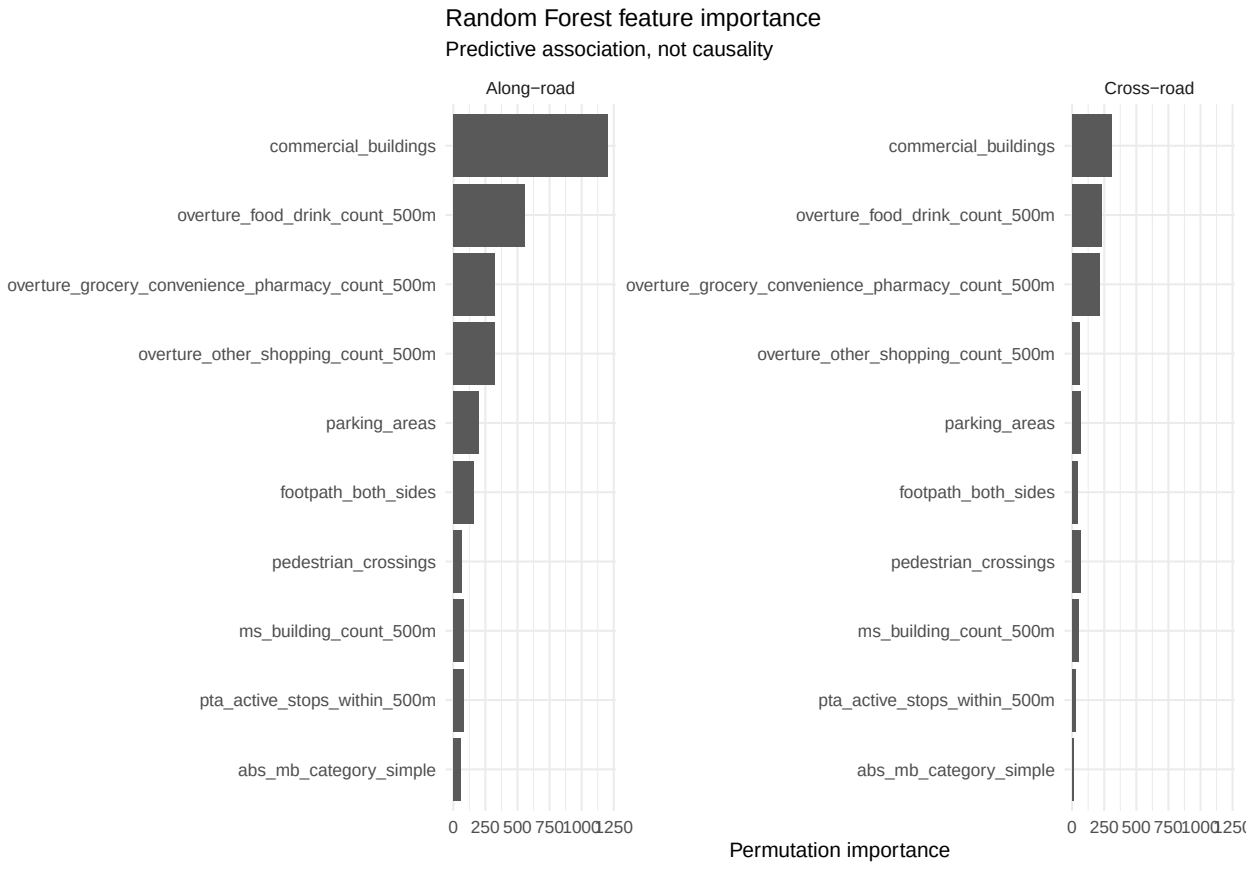
kable(
  rf_top,
  digits = 3,
  caption = "Top Random Forest permutation importances"
)
```

Table 15: Top Random Forest permutation importances

Target	Feature	Importance
Along-road	commercial_buildings	1205.859
Along-road	overture_food_drink_count_500m	559.746
Along-road	overture_grocery_convenience_pharmacy_count_500m	324.427
Along-road	overture_other_shopping_count_500m	319.522
Along-road	parking_areas	198.797
Along-road	footpath_both_sides	156.617
Along-road	ms_building_count_500m	80.061
Along-road	pta_active_stops_within_500m	78.882
Along-road	pedestrian_crossings	66.599
Along-road	abs_mb_category_simple	57.104
Cross-road	commercial_buildings	307.131

Target	Feature	Importance
Cross-road	overture_food_drink_count_500m	229.646
Cross-road	overture_grocery_convenience_pharmacy_count_500m	214.980
Cross-road	parking_areas	69.574
Cross-road	pedestrian_crossings	67.262
Cross-road	overture_other_shopping_count_500m	62.029
Cross-road	ms_building_count_500m	48.432
Cross-road	footpath_both_sides	42.492
Cross-road	pta_active_stops_within_500m	27.066
Cross-road	abs_mb_category_simple	10.871

```
ggplot(
  rf_top,
  aes(x = reorder(Feature, Importance), y = Importance)
) +
  geom_col() +
  coord_flip() +
  facet_wrap(~ Target, scales = "free_y") +
  labs(
    title = "Random Forest feature importance",
    subtitle = "Predictive association, not causality",
    x = NULL,
    y = "Permutation importance"
  )
```



26 25. Gradient Boosting feature importance

```
gb_importance <- function(fit, target_label) {

  imp <- summary(fit$model, plotit = FALSE)

  tibble(
    Target = target_label,
    Feature = imp$var,
    Importance = imp$rel.inf
  ) %>%
    arrange(desc(Importance))
}

gb_imp <- bind_rows(
  gb_importance(final_cross$gradient_boosting, "Cross-road"),
  gb_importance(final_along$gradient_boosting, "Along-road")
)

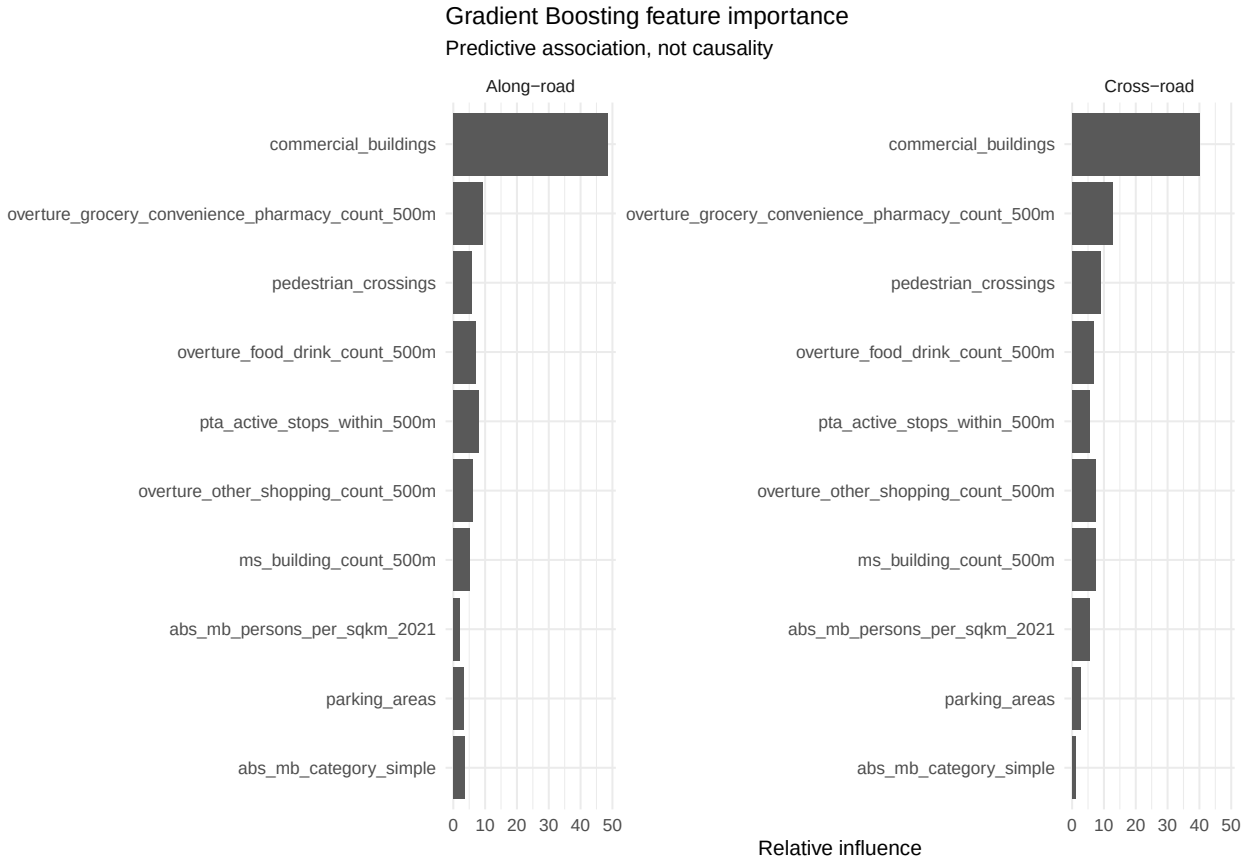
gb_top <- gb_imp %>%
  group_by(Target) %>%
  slice_max(Importance, n = 10, with_ties = FALSE) %>%
  ungroup()

kable(
  gb_top,
  digits = 3,
  caption = "Top Gradient Boosting relative influences"
)
```

Table 16: Top Gradient Boosting relative influences

Target	Feature	Importance
Along-road	commercial_buildings	48.643
Along-road	overture_grocery_convenience_pharmacy_count_500m	9.117
Along-road	pta_active_stops_within_500m	7.960
Along-road	overture_food_drink_count_500m	7.063
Along-road	overture_other_shopping_count_500m	6.137
Along-road	pedestrian_crossings	5.966
Along-road	ms_building_count_500m	5.224
Along-road	abs_mb_category_simple	3.546
Along-road	parking_areas	3.241
Along-road	abs_mb_persons_per_sqkm_2021	1.967
Cross-road	commercial_buildings	39.962
Cross-road	overture_grocery_convenience_pharmacy_count_500m	12.885
Cross-road	pedestrian_crossings	9.082
Cross-road	ms_building_count_500m	7.479
Cross-road	overture_other_shopping_count_500m	7.246
Cross-road	overture_food_drink_count_500m	6.765
Cross-road	pta_active_stops_within_500m	5.593
Cross-road	abs_mb_persons_per_sqkm_2021	5.464
Cross-road	parking_areas	2.804
Cross-road	abs_mb_category_simple	1.233

```
ggplot(
  gb_top,
  aes(x = reorder(Feature, Importance), y = Importance)
) +
  geom_col() +
  coord_flip() +
  facet_wrap(~ Target, scales = "free_y") +
  labs(
    title = "Gradient Boosting feature importance",
    subtitle = "Predictive association, not causality",
    x = NULL,
    y = "Relative influence"
  )
```



27 26. Dashboard-ready out-of-fold predictions

These predictions are appropriate for evaluation visualisations because every value is out of fold.

```
dashboard_output <- all_predictions %>%
  select(
    Target, Model,
    site_id, road_no, road_name,
    actual, predicted,
    residual, absolute_error
  )
```

```r
dir.create(
  "../06_outputs/03_model_development/03_yashika_model_comparison",
  showWarnings = FALSE,
  recursive = TRUE
)

write.csv(
  dashboard_output,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_model_comparison_oof_predictions.cs
  row.names = FALSE
)

write.csv(
  cv_performance,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_model_comparison_cv_metrics.csv",
  row.names = FALSE
)

write.csv(
  overall_ranking,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_model_comparison_overall_ranking.cs
  row.names = FALSE
)

write.csv(
  bootstrap_ci,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_model_comparison_bootstrap_ci.csv"
  row.names = FALSE
)

write.csv(
  largest_errors,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_model_comparison_largest_errors.csv
  row.names = FALSE
)

kable(
  head(
    dashboard_output %>% filter(Model == selected_model),
    12
  ),
  digits = 2,
  caption = paste(
    "Example out-of-fold predictions from",
    selected_model
  )
)
```

Table 17: Example out-of-fold predictions from Linear regression

Target	Model	site_id	road_no	road_name	actual	predicted	residual	absolute_error
Cross-road	Linear regression	250303-01	M074	Melville Mandurah Hwy	0.33	2.98	-2.65	2.65
Cross-road	Linear regression	250303-02	H009	South Western Hwy	4.00	9.50	-5.50	5.50
Cross-road	Linear regression	250303-03	H045	Coalfields Hwy	47.00	30.36	16.64	16.64
Cross-road	Linear regression	250303-04	H009	South Western Hwy	16.00	2.22	13.78	13.78
Cross-road	Linear regression	250303-05	H009	South Western Hwy	57.33	37.45	19.89	19.89
Cross-road	Linear regression	250303-06	H009	South Western Hwy	52.00	4.16	47.84	47.84
Cross-road	Linear regression	250303-07	H009	South Western Hwy	46.67	37.45	9.22	9.22
Cross-road	Linear regression	250303-08	M042	Goodwood Rd	34.33	30.61	3.73	3.73
Cross-road	Linear regression	250303-09	H009	South Western Hwy	26.33	28.54	-2.21	2.21
Cross-road	Linear regression	250303-10	H009	South Western Hwy	96.67	107.14	-10.47	10.47
Cross-road	Linear regression	250303-11	H043	Bussell Hwy	192.33	89.41	102.92	102.92
Cross-road	Linear regression	250303-12	H043	Bussell Hwy	52.00	38.31	13.69	13.69

28 27. Save the final model bundle

```
model_bundle <- list(
  input_file = data_path,
  features = features,
  numeric_features = numeric_features,
  binary_features = binary_features,
  categorical_features = categorical_features,
  linear_features = linear_features,
  targets = targets,
  grouped_cv_performance = cv_performance,
  overall_ranking = overall_ranking,
  selected_model = selected_model,
  runner_up_model = runner_up_model,
  bootstrap_intervals = bootstrap_ci,
  bootstrap_selected_vs_runner_up = selected_runner_diff,
  final_models = list(
    Cross = final_cross,
    Along = final_along
  ),
  seed = 5553
)

saveRDS(
```

```
  model_bundle,
  "../06_outputs/03_model_development/03_yashika_model_comparison/03_pedestrian_flow_final_model_bundle
)
```

29 28. Final conclusion

```
best_results <- cv_performance %>%
  filter(Model == selected_model) %>%
  arrange(Target)

cat(
  "The final model comparison used road-grouped five-fold cross-validation, ",
  "so each reported prediction was generated for a road excluded from model training. ",
  "This design is more appropriate than relying on training fit for the small survey dataset.\n\n",
  sep = ""
)
```

The final model comparison used road-grouped five-fold cross-validation, so each reported prediction was generated for a road excluded from model training. This design is more appropriate than relying on training fit for the small survey dataset.

```
cat(
  "**", selected_model, "** achieved the strongest aggregate out-of-sample ranking ",
  "across the two pedestrian-flow targets when R-squared, RMSE and MAE were considered together.\n\n",
  sep = ""
)
```

Linear regression achieved the strongest aggregate out-of-sample ranking across the two pedestrian-flow targets when R-squared, RMSE and MAE were considered together.

```
for (i in seq_len(nrow(best_results))) {
  x <- best_results[i, ]

  cat(
    "- **", x$Target, "**: R-squared = ", sprintf("%.3f", x$R2),
    ", RMSE = ", sprintf("%.2f", x$RMSE),
    ", MAE = ", sprintf("%.2f", x$MAE),
    " pedestrians per hour.\n",
    sep = ""
  )
}
```

- **Along-road**: R-squared = 0.719, RMSE = 37.02, MAE = 20.26 pedestrians per hour.
- **Cross-road**: R-squared = 0.425, RMSE = 29.14, MAE = 15.18 pedestrians per hour.

```
cat(
  "\nRoad-level bootstrapping with 1,000 resamples was used to quantify uncertainty ",
  "in the observed performance. Feature-importance results should be interpreted as ",
  "predictive associations rather than causal effects.\n\n",
  sep = ""
)
```

Road-level bootstrapping with 1,000 resamples was used to quantify uncertainty in the observed performance. Feature-importance results should be interpreted as predictive associations rather than causal effects.

```r
cat(
  "Because the current survey dataset is small, this should be treated as a baseline ",
  "deployment model rather than a final universal model. The highest-priority next step ",
  "is to expand survey coverage, especially for high-flow sites and roads not represented ",
  "in the present sample, and then repeat the grouped validation on the expanded dataset.\n",
  sep = ""
)
```

Because the current survey dataset is small, this should be treated as a baseline deployment model rather than a final universal model. The highest-priority next step is to expand survey coverage, especially for high-flow sites and roads not represented in the present sample, and then repeat the grouped validation on the expanded dataset.

30 29. Key limitations

1. **Small sample size.** Model estimates can change materially when only a few new survey sites are added.
2. **Limited road coverage.** Grouped validation reduces leakage, but external validation on genuinely new roads would be stronger.
3. **High-flow observations.** A small number of very busy sites can dominate RMSE.
4. **Correlated commercial features.** Retail and commercial-density variables partly measure the same underlying activity.
5. **Feature importance is not causal.** Important predictors identify useful predictive signals, not necessarily causes of pedestrian movement.
6. **Temporal coverage is limited.** Available surveys may not fully capture seasonal or time-of-day variation.
7. **Deployment shift is possible.** Full-network prediction may encounter road environments not represented in the survey sample.

31 30. Recommended next steps

1. Add more pedestrian survey sites, prioritising new roads and high-flow environments.
2. Re-run the same grouped validation after each major data update.
3. Preserve Overture feature definitions and ETL logic so model inputs remain reproducible.
4. Compare predicted flow with existing iRAP exposure values and survey counts in the dashboard.
5. Flag road segments whose predictor values fall outside the range represented by the training data.
6. Treat network-wide predictions as estimates with uncertainty, not direct measurements.

32 31. What to show in the next team/client meeting

Show these outputs in this order:

1. **Dataset and grouped-fold summary** — demonstrates leakage-safe validation.
2. **Cross-validated performance table** — compares all six approaches fairly.
3. **Overall model ranking** — gives a transparent reason for the selected model.
4. **Actual-versus-predicted plots** — shows where each model succeeds and fails.
5. **Improvement over the median baseline** — demonstrates practical predictive value.
6. **Road-level bootstrap intervals** — communicates uncertainty.
7. **Selected-model versus runner-up bootstrap comparison** — tests whether the apparent advantage is stable.
8. **Feature-importance / coefficient outputs** — explains the predictive signals.
9. **Largest prediction errors** — demonstrates critical evaluation rather than only favourable results.
10. **Dashboard-ready output** — connects modelling directly to the system deliverable.

The central message is:

> **The final model is selected using out-of-sample evidence on unseen roads, not training fit. Multiple statistical and machine-learning approaches are evaluated under the same grouped cross-validation design, and road-level bootstrapping is used to quantify the stability of the observed performance.**